

[CHECK VALUE: final thesis title]

PC-hosted n8n, Python middleware, shared contracts, and
Tier 1.5 Raspberry Pi validation

[CHECK VALUE: author name]

[CHECK VALUE: DT099G thesis project type]

Main field of study: Computer Engineering

Credits: 15 hp

Semester, Year: Spring, 2026

Supervisor: [CHECK VALUE: supervisor name]

Examiner: [CHECK VALUE: examiner name]

Course code/Registration number: DT099G/ [CHECK VALUE: registration number]

Programme: [CHECK VALUE: study programme or course]

At Mid Sweden University, it is possible to publish the thesis in full text in DiVA (see appendix for publishing conditions). The publication is open access, which means that the work will be freely available to read and download online. This increases the dissemination and visibility of the degree project.

Open access is becoming the norm for disseminating scientific information online. Mid Sweden University recommends both researchers and students to publish their work open access.

I/we allow publishing in full text (free available online, open access):

- ☐ Yes, I/we agree to the terms of publication.
- ☐ No, I/we do not accept that my independent work is published in the public interface in DiVA (only archiving in DiVA).

.....
Location and date

.....
Programme/Course

.....
Name (all authors names)

.....
Year of birth (all authors year of birth)

Abstract

[CHECK VALUE: write final 200-250 word English abstract after Chapters 3-7 are drafted.]

Keywords: [CHECK VALUE: final keywords, for example n8n, workflow automation, IoT, middleware, JSON contract, Raspberry Pi]

Sammanfattning

[CHECK VALUE: write Swedish summary after the English abstract is complete.]

Nyckelord: [CHECK VALUE: Swedish keyword list]

Acknowledgements/Foreword

Contents

Acknowledgements	iv
List of Figures	viii
List of Tables	ix
List of Algorithms	x
Terminology / Notation	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Aim	2
1.4 Research Questions	2
1.5 Scope and Delimitations	3
1.6 Contributions	3
1.7 Use of AI Tools	4
1.8 Report Outline	4
2 Theory and Related Work	5
2.1 Workflow Automation and Orchestration	5
2.2 IoT Event-Action Systems and Edge Computing	5
2.3 Middleware and API Boundaries	6
2.4 Structured Data, JSON Contracts, and Validation	6
2.5 Deterministic Workflows	7
2.6 Minimal Agent-Enhanced Workflows	7
2.7 Safety, Validation, and Human-in-the-Loop Concepts	7
2.8 Summary of Theory	8
3 Methodology	9
3.1 Research Approach	9
3.2 Development Method	9
3.3 Evaluation Design	10
3.4 Metrics and Data Collection	11
3.5 Raspberry Pi Validation Method	11
3.6 Validity and Limitations of the Method	12
3.7 Summary of Method	12
4 Choice of Approach / System Design	14
4.1 Design Requirements and Constraints	14
4.2 Alternatives Considered	14
4.3 Chosen System Design	15

4.4	Rationale for PC-Hosted n8n and Raspberry Pi Middle-ware Validation	17
4.5	Rationale for Deterministic and Agent-Enhanced Paths	18
4.6	Rationale for Middleware, Contracts, and Validation	18
4.7	Rationale for Simulated Fan Actions	19
4.8	Rationale for Lightweight Evaluation	19
4.9	Summary of Chosen Approach	19
5	Implementation	21
5.1	System Overview	21
5.2	Local n8n Runtime with Docker	22
5.3	Python Middleware API Bridge	23
5.4	Shared JSON Contracts	24
5.5	Deterministic Baseline Workflow	24
5.6	Minimal Agent-Enhanced Workflow	25
5.7	Minimum Safety and Validation Design	26
5.8	Evaluation Harness	27
5.9	Raspberry Pi Tier 1.5 Validation Setup	28
5.10	Summary of Implementation	28
6	Results	30
6.1	Overview of Evaluated System	30
6.2	Deterministic Baseline Results	30
6.3	Minimal Agent-Enhanced Workflow Results	31
6.4	Safety and Validation Case Results	32
6.5	Evaluation Harness Outputs	32
6.6	Raspberry Pi Tier 1.5 Validation Results	34
6.7	Summary of Results	36
7	Discussion	37
7.1	Limitations of the Current Implementation	37
7.2	Discussion of RQ1: Deterministic Workflow Accuracy and Consistency	38
7.3	Discussion of RQ2: Agent-Enhanced Contract-Valid Out-put	38
7.4	Discussion of RQ3: Latency, Resource, and Deploy-ment Behavior	39
7.5	Role of Middleware, Contracts, and Validation	40
7.6	Broader Relevance and Possible Use Cases	41
7.7	Ethical and Societal Considerations	41
7.8	Future Work	42
7.9	Summary of Discussion	43
8	Conclusions	44

8.1	Conclusions	44
8.2	Main Contributions	45
8.3	Future Work	45
8.4	Final Remarks	46
References		47
A	Source Code and Artifacts	1
A.1	Repository Reference	1
A.2	Workflow and Screenshot Evidence	1
A.3	Evaluation Artifacts	1

List of Figures

1	Final focused proof-of-concept architecture used in the thesis.	17
2	Frozen Tier 1.5 proof-of-concept architecture.	22
3	Deterministic baseline n8n workflow canvas.	25
4	Minimal agent-enhanced n8n workflow canvas.	26
5	Raspberry Pi Tier 1.5 validation output for the high-temperature fan-on case.	28
6	Raspberry Pi Tier 1.5 validation output for the high-temperature fan-on case.	36

List of Tables

1	Planned terminology entries to finalize during drafting. . .	xi
2	Defined workflow test cases from evaluation/datasets/test-cases.json. 10	
3	Deterministic baseline results from evaluation/results/raw/run_20260429T165733Z_deterministic.csv. . . .	30
4	Minimal agent-enhanced workflow results from evaluation/results/raw/run_20260429T165859Z_agent.csv. .	31
5	Documented safety and validation cases from Step 8. . .	32
6	Evaluation harness output files.	33
7	Final workflow latency summary.	34
8	Raspberry Pi Tier 1.5 full-workflow validation from pi-workflow-run-final.csv.	35
9	Supporting Raspberry Pi middleware observations. . . .	35

List of Algorithms

Terminology/Notation

Table 1: Planned terminology entries to finalize during drafting.

Term	Meaning to define in the final report
API	[CHECK VALUE: define consistently with middleware implementation.]
CSV	[CHECK VALUE: define as the evaluation result format.]
GPIO	[CHECK VALUE: explain that real GPIO control was not implemented.]
HITL	[CHECK VALUE: define human-in-the-loop approval as a minimum design element.]
JSON contract	[CHECK VALUE: define using shared interface schemas.]
LLM	[CHECK VALUE: define only as used by the minimal agent-enhanced workflow.]
n8n	[CHECK VALUE: define as the self-hosted workflow automation engine.]
Tier 1.5 validation	[CHECK VALUE: define as Raspberry Pi middleware/action validation while n8n remains on PC.]

1 Introduction

This thesis investigates how a workflow automation system can be used as part of a small, measurable IoT-style action pipeline. The work focuses on a controlled temperature-to-fan scenario in which a workflow receives a temperature event, selects a fan action, passes the action through a structured contract boundary, calls Python middleware, and records evaluation evidence. The purpose is not to build a production IoT platform, but to examine whether a focused workflow-to-action architecture can be implemented and evaluated clearly.

1.1 Background and Motivation

Workflow automation platforms are often used to connect triggers, decision logic, external services, and actions. In such systems, a workflow can receive an event, process it through configured nodes, and call an external endpoint. n8n is one such workflow automation platform and can be self-hosted for local experimentation and integration work [1], [2]. This makes it suitable for a thesis prototype where the workflow engine should be visible, configurable, and testable.

IoT-style systems often follow an event-action pattern: a sensor or test event is received, a decision is made, and an action is triggered. Edge hardware such as Raspberry Pi can also be used to place part of a system closer to a device or environment [3], [4], [5]. However, when workflows or agent-enhanced workflow steps are allowed to trigger actions, the system needs more than a decision rule. It also needs a controlled action boundary, structured messages, validation, and evidence that the expected action was actually produced.

This thesis uses a fan on/off scenario as a controlled proxy for that broader problem. The fan itself is simple, but the architecture around it is the important part: a temperature event enters the workflow, the workflow selects or produces an action, the action is represented in a structured form, middleware executes a simulated endpoint, and the evaluation harness records the result. This allows the thesis to study a complete event-to-action path without expanding into a broad hardware or automation platform.

1.2 Problem Statement

The problem addressed in this thesis is that an IoT-style workflow is not sufficiently evaluated by showing only that a workflow can make a decision. If a workflow or agent-enhanced step can trigger an action, the system also needs a clear boundary between workflow logic and action execution. Without such a boundary, it becomes difficult to explain which component made the decision, which component executed the action, and what structure the action message was expected

to follow.

The thesis therefore treats the problem as an integration and evaluation problem. The system needs structured action messages, a documented validation design, a middleware layer that separates workflow decisions from action execution, CSV-based evidence for result reporting, and limited hardware-side validation. JSON contracts are used to make the sensor and action payloads explicit [6]. The problem is investigated within a controlled proof-of-concept rather than through a large-scale deployment.

1.3 Aim

The aim of this thesis is to design, implement, and evaluate a self-hosted n8n-based proof-of-concept for a temperature-to-fan IoT-style action pipeline using Python middleware, structured contracts, lightweight evaluation, and Raspberry Pi middleware validation.

The aim is addressed by implementing a deterministic baseline workflow, a minimal agent-enhanced workflow, shared JSON contracts, a minimum safety and validation design, simulated fan endpoints, a CSV evaluation harness, and a validation setup where the middleware/action endpoint runs on Raspberry Pi while n8n remains on the PC.

1.4 Research Questions

The thesis is guided by three research questions:

1. **RQ1:** How accurately and consistently does the deterministic n8n workflow produce the expected fan action across the defined temperature test cases?
2. **RQ2:** How accurately and consistently does the minimal agent-enhanced workflow produce contract-valid action output compared with the deterministic baseline?
3. **RQ3:** What latency, resource, and deployment behavior is observed during local execution and Raspberry Pi middleware validation?

No separate yes/no research question is created for safety. Instead, contract validity and the documented validation boundary are evaluated as part of RQ2 and discussed further in Chapter 7. This keeps the research questions measurable and aligned with the implemented system.

1.5 Scope and Delimitations

The defined scope includes a local PC-hosted n8n runtime using Docker, a Python middleware API, one temperature-event path, simulated `fan_on` and `fan_off` actions, a deterministic baseline workflow, a minimal agent-enhanced workflow, shared JSON contracts, a documented minimum safety and validation design, a CSV-based evaluation harness, and Raspberry Pi middleware/action endpoint validation. The middleware exposes `/status`, `/sensor-event`, `/fan/on`, and `/fan/off`.

The system is deliberately kept as a controlled proof-of-concept. Full n8n deployment on Raspberry Pi is outside the scope. Real GPIO fan or sensor hardware is outside the scope. Production-grade runtime safety enforcement is outside the scope. Broad multi-agent architecture, multiple models, multiple devices, and large-scale benchmarking are also outside the scope. These delimitations allow the thesis to focus on one complete and measurable workflow-to-action pipeline.

1.6 Contributions

The technical contribution of the thesis is a complete temperature-to-fan proof-of-concept architecture that combines self-hosted n8n workflow orchestration, a Python middleware action boundary, deterministic and minimal agent-enhanced workflow paths, shared JSON contracts, a documented validation design, CSV-based evaluation, and Raspberry Pi middleware validation. The contribution is not a new IoT platform or a production safety system. It is a focused implementation and evaluation of a workflow-to-action pattern that can be inspected, measured, and discussed.

The author contributions are summarized using a concise CRediT-style structure:

- Conceptualization: author, with supervisor feedback.
- Methodology: author, with supervisor feedback.
- Software: author.
- Validation: author.
- Investigation: author.
- Data curation: author.
- Writing – original draft: author.
- Writing – review and editing: author, with supervisor feedback.
- Supervision: supervisor.

1.7 Use of AI Tools

AI tools were used as support during the thesis work for planning, coding assistance, debugging support, report structuring, and language drafting. These tools were used as assistants, not as authorities. The author remains responsible for the implementation decisions, verification, result interpretation, references, and final report content. No implementation behavior, measurement value, source file, or result is treated as valid only because an AI tool suggested it; such material must be supported by repository evidence or cited sources.

1.8 Report Outline

Chapter 2 presents the theory and related work needed to understand workflow automation, IoT-style event-action systems, contracts, agent-enhanced workflows, validation, and edge hardware context. Chapter 3 describes the research approach and evaluation method. Chapter 4 explains the choice of approach and justifies the final system design. Chapter 5 describes the implemented proof-of-concept. Chapter 6 reports the observed results from the deterministic workflow, agent-enhanced workflow, evaluation harness, and Raspberry Pi validation. Chapter 7 interprets the results, answers the research questions, discusses limitations, and connects the fan scenario to broader use cases. Chapter 8 presents the final conclusions and future work. The appendix provides references to source code and supporting artifacts.

2 Theory and Related Work

This chapter presents the theoretical background needed to understand the proof-of-concept developed in this thesis. The focus is on workflow automation, IoT-style event-action systems, middleware boundaries, structured contracts, deterministic and agent-enhanced workflows, validation, and limited edge-hardware validation. The chapter does not attempt to survey all IoT platforms, all workflow engines, or all agent architectures. Instead, it introduces the concepts that are needed to justify and interpret the implemented system.

2.1 Workflow Automation and Orchestration

Workflow automation concerns the coordination of tasks, decisions, and integrations through an explicit process structure. A workflow can start from an event, apply decision logic, transform data, and call external systems. This makes workflow automation useful in event-action systems where the purpose is not only to compute a value, but to connect an incoming event to a controlled response. Workflow patterns are a well-established way of describing recurring control-flow structures such as branching, routing, and synchronization [2].

In this thesis, workflow automation is used as an orchestration layer. The workflow engine receives a temperature event, evaluates the event, and calls middleware endpoints. n8n is the workflow platform used for this purpose. It provides a self-hostable workflow environment with nodes for triggers, data handling, and HTTP calls [1]. The choice of n8n is not treated as a comparison against other workflow tools; it is the concrete orchestration platform used to implement and evaluate the thesis architecture.

2.2 IoT Event-Action Systems and Edge Computing

IoT systems commonly involve devices, sensors, network communication, data processing, and actions in the physical or operational environment. A sensor event may represent a temperature reading, a state change, or another observation that requires a response. The response may be an alert, a control action, or a decision to do nothing. This event-action pattern is central to the thesis scenario, where a temperature event is used to select a simulated fan action. Broader IoT research identifies this kind of connection between sensing, computation, and action as a core part of Internet of Things systems [4].

Edge computing is relevant when some processing or action endpoint is moved closer to the device or environment rather than keeping all functionality in a central server or cloud system. Edge computing

can reduce reliance on distant infrastructure and can support latency-sensitive or context-specific behavior [3]. In this thesis, Raspberry Pi is used only as limited edge-style hardware for middleware/action endpoint validation. Raspberry Pi is a small computing platform that can run software services and connect to hardware interfaces [5]. The thesis does not evaluate a complete edge deployment; it uses Raspberry Pi to validate the middleware/action side of the architecture.

2.3 Middleware and API Boundaries

Middleware is useful when a system needs to separate decision logic from action execution. In an event-action architecture, the workflow should not necessarily be the component that directly controls hardware-facing behavior. A middleware/API layer can receive a structured action request, expose a small set of endpoints, and isolate workflow logic from the internal details of the action implementation. This separation makes the system easier to reason about because the workflow, contract, validation design, and action endpoint can be discussed as separate parts of the architecture [7], [8].

In this thesis, the middleware/API boundary is especially important because both deterministic workflow output and agent-enhanced workflow output may lead to fan actions. Routing those actions through middleware prevents the workflow or agent step from being treated as direct hardware control. It also creates a location where simulated endpoints can later be replaced by physical device control, provided that additional validation and safety work is added.

2.4 Structured Data, JSON Contracts, and Validation

Structured data formats reduce ambiguity between connected components. If a workflow emits only free text, the receiving system must infer the intended action. If the workflow emits a structured object, the receiving system can inspect fields, allowed values, and missing data more consistently. This is why the thesis uses JSON-style sensor and action contracts rather than relying on informal text output.

JSON Schema provides a vocabulary for describing and validating the structure of JSON data [6]. At a theory level, a schema can define required fields, allowed values, and whether unexpected fields are permitted. In an event-action system, this makes it possible to describe what a valid sensor event or action request should look like. In this thesis, structured contracts support predictable routing and provide a basis for discussing validation. They do not by themselves guarantee production safety, but they make the intended boundary explicit.

2.5 Deterministic Workflows

A deterministic workflow uses fixed logic to map inputs to outputs. Given the same input and the same workflow configuration, the expected decision path is the same. In the thesis scenario, the deterministic baseline uses a temperature threshold: a value at or above the threshold leads to `fan_on`, while a lower value leads to `fan_off`.

Deterministic workflows are useful as baselines because their behavior is transparent and inspectable. The rule can be stated directly, the expected output can be defined before execution, and deviations can be identified clearly. This makes the deterministic baseline important for evaluating the rest of the proof-of-concept. It shows whether the basic workflow, middleware, action endpoints, and evaluation harness work for the defined temperature cases before the agent-enhanced path is considered.

2.6 Minimal Agent-Enhanced Workflows

An agent-enhanced workflow includes a decision step where a language model contributes to selecting or producing an action. In broader research, language-model agents may combine reasoning and acting by producing intermediate reasoning steps, using tools, and interacting with external environments [9]. This thesis uses a much smaller version of that idea. The agent-enhanced path contains a single LLM-based decision step that is expected to produce structured action output for the same temperature-to-fan scenario as the deterministic baseline.

The important theoretical point is not broad autonomy. The important point is that an LLM decision step can be placed inside a workflow, constrained by a prompt, and connected to a structured output pipeline. For this thesis, the output is expected to fit the action contract and route to the same middleware endpoints as the deterministic baseline. This allows the agent-enhanced workflow to be compared with the deterministic workflow without changing the action execution layer.

2.7 Safety, Validation, and Human-in-the-Loop Concepts

When workflow or agent output can trigger actions, validation becomes important. A system should be able to distinguish between an allowed action, a malformed action, and a risky or unsupported action. In the thesis scenario, allowed actions are limited to the simulated fan actions, while malformed or unknown actions should not be treated as valid action requests.

Human-in-the-loop concepts are relevant when automation may require review before execution. Human involvement can help with cases where automated output is uncertain, risky, or outside the expected action set. Research on interactive machine learning emphasizes the role of humans in guiding and controlling machine-assisted systems [10]. In this thesis, the safety material remains a documented validation and approval design. It supports reasoning about allowed, blocked, and risky cases, but it is not evidence of production-grade runtime enforcement.

2.8 Summary of Theory

The theory in this chapter supports the thesis architecture as a combination of workflow automation, IoT-style event-action logic, middleware separation, structured JSON contracts, deterministic baseline behavior, a constrained agent-enhanced decision step, validation concepts, and limited Raspberry Pi middleware validation. Together, these concepts explain why the implemented system is structured as a controlled workflow-to-action proof-of-concept rather than as a broad IoT platform or general multi-agent system.

3 Methodology

This chapter describes how the thesis work was carried out and how the proof-of-concept system was evaluated. The method was chosen to match the narrowed scope of the thesis: one controlled event-action scenario, two workflow decision modes, a shared contract boundary, a minimum safety and validation design, lightweight CSV-based measurement, and limited Raspberry Pi validation.

3.1 Research Approach

The thesis uses a design-and-evaluate proof-of-concept approach. The work first defines a limited technical problem, then builds an artifact that addresses that problem, evaluates the artifact with defined test cases, and finally discusses the limitations of the evidence. This approach is suitable for the thesis because the research questions are not answered only by theoretical comparison; they require an implemented system that can produce observable workflow behavior, action outputs, latency values, resource observations, and deployment evidence [11], [12].

The artifact evaluated in this thesis is the narrowed workflow system: PC-hosted Docker n8n, Python middleware, deterministic and minimal agent-enhanced workflow paths, shared JSON contracts, a documented safety and validation boundary, a CSV evaluation harness, and Raspberry Pi middleware/action endpoint validation. The evaluation is therefore artifact-centered and scenario-based. It does not attempt to benchmark many platforms, many devices, or many language models.

3.2 Development Method

The implementation was developed through controlled stages. This staged method was used to reduce scope risk and keep each part of the system testable before adding the next one. The process started with a scope lock, where the broader historical thesis direction was narrowed to one measurable infrastructure-focused proof-of-concept. After that, the local n8n runtime was established on the PC, followed by the Python middleware, shared JSON contracts, the deterministic workflow, the minimal agent-enhanced workflow, the minimum safety design, the evaluation harness, Raspberry Pi validation, and finally an implementation freeze.

This sequence was not intended as a diary of implementation events. It was used as a controlled development method: each stage introduced one necessary layer and produced evidence that could be used in the report. The n8n runtime established the workflow execution environment. The middleware provided the action boundary. The shared con-

tracts made the event and action payloads explicit. The deterministic workflow provided a baseline. The minimal agent-enhanced workflow introduced a constrained agent-style decision step. The safety design documented the allowed, blocked, and risky action cases. The evaluation harness made the system measurable through CSV files. Raspberry Pi validation then tested a limited split-deployment case without changing the core PC-first architecture.

The final implementation freeze was used to prevent additional features from expanding the scope after the system had enough evidence for report writing. The freeze explicitly deferred full n8n-on-Raspberry-Pi deployment, real GPIO hardware, broader multi-agent behavior, production-grade safety enforcement, MCP integration, multiple devices, multiple models, and heavy observability stacks.

3.3 Evaluation Design

The evaluation was designed around one temperature-to-fan scenario. The same input cases were used for the deterministic baseline workflow and the minimal agent-enhanced workflow so that the two modes could be compared on the same task. The evaluation also included documented safety and validation cases and a separate Raspberry Pi validation setup.

The deterministic baseline was evaluated by checking whether the fixed n8n threshold rule produced the expected fan action for the defined high- and low-temperature cases. The minimal agent-enhanced workflow was evaluated by checking whether the agent path produced the expected fan action and whether the output followed the shared action contract shape. The safety and validation cases were evaluated as documented design cases: one allowed action, one malformed blocked action, and one risky or unsupported action that should not be executed directly. The Raspberry Pi validation evaluated whether PC-hosted n8n could call a Raspberry Pi-hosted middleware/action endpoint for the same temperature-to-fan scenario.

Table 2: Defined workflow test cases from evaluation/datasets/test-cases.json.

Test case type	Input temperature	Expected action	Mode
High-temperature workflow case	31.4 C	fan_on	Deterministic and agent
Low-temperature workflow case	24.5 C	fan_off	Deterministic and agent

The safety cases in the dataset use action objects rather than temperature events. The allowed case contains a complete `fan_on` action for `fan_1`. The blocked case omits required fields. The risky case uses an unsupported `open_window` action and an unknown target. These safety cases support evaluation of the documented validation boundary, but they do not by themselves prove production runtime enforcement.

3.4 Metrics and Data Collection

The evaluation records both behavioral and measurement-oriented fields. The behavioral fields include the expected action, observed action, success or failure, expected safety result, and observed safety result where applicable. The measurement fields include latency, CPU values where available, RAM values where available, and thermal observations where available. Contract validity is considered where workflow or agent output is expected to follow the shared action object structure.

Data collection is CSV-based. Raw result rows are written under `evaluation/results/raw/`, and processed summaries are written under `evaluation/results/processed/`. The script `scripts/collect_metrics.py` posts test payloads to the configured workflow webhook and records the raw result fields. For workflow modes, success is recorded when the HTTP request succeeds and the observed action matches the expected action. The script measures latency using Python timing around the webhook HTTP request. It also attempts best-effort CPU and RAM collection through `docker stats --no-stream` when Docker statistics are available [13]. Thermal values are not fabricated; when no supported thermal source is available, the field is recorded as `not_available`.

The script `scripts/aggregate_results.py` aggregates raw CSV files into processed summaries. The defined processed outputs are `summary_latency.csv`, `summary_resources.csv`, `safety_outcomes.csv`, and `baseline_vs_agent.csv`. CSV is used as a simple tabular exchange format, with Python's standard `csv` module used by the scripts [14], [15]. These files support the Results chapter by separating raw observations from summarized comparison tables. The evaluation protocol also notes that safety-mode rows can document expected safety behavior without proving runtime enforcement unless enforcement logic is implemented and logged.

3.5 Raspberry Pi Validation Method

Raspberry Pi validation was used to test the middleware/action endpoint on separate hardware, while `n8n` remained running on the PC. The PC-hosted `n8n` workflow called the Raspberry Pi middleware over HTTP.

This split validation method was chosen to test part of the edge deployment path without changing the frozen architecture. It evaluates whether the middleware/action endpoint can be moved to the Raspberry Pi while preserving the PC-hosted n8n workflow. It does not evaluate full n8n-on-Pi deployment, and it does not claim that the whole system ran on the Raspberry Pi. The fan action remained a simulated middleware action, and no real GPIO sensor or physical fan was controlled.

The Raspberry Pi evidence was stored under `evaluation/results/pi-validation/`. The method included the same high- and low-temperature fan cases, full workflow latency collection for the PC-to-Pi workflow path, supporting direct endpoint latency observations, CPU/RAM observations for the middleware process, and thermal observation from the Raspberry Pi evidence.

3.6 Validity and Limitations of the Method

The method has several limitations that affect how the results should be interpreted. The number of test cases is small because the thesis focuses on one controlled temperature-to-fan scenario. The run count is also limited, so the results should be treated as proof-of-concept evidence rather than broad benchmarking. The final report therefore uses the data to evaluate whether the defined system works for the defined cases, not to claim general performance across many environments.

The fan action is simulated in middleware. No real GPIO sensor, real fan, or physical actuator was controlled. This limits claims about hardware behavior, electrical reliability, and physical safety. The safety layer is also limited: it is a documented design and specification layer with allowed, blocked, and risky cases. It is not a production-grade runtime safety framework, and expected safety behavior should not be reported as runtime enforcement unless corresponding runtime evidence exists.

The Raspberry Pi validation is similarly limited. It validates only the middleware/action endpoint side while n8n remains on the PC. This supports discussion of edge-style deployment behavior, but it is not evidence of full edge deployment and not evidence of n8n performance on Raspberry Pi. Local PC thermal collection is also limited because thermal values are recorded only where a supported source exists.

3.7 Summary of Method

The method supports the research questions by combining a controlled implementation artifact with repeatable, file-based evaluation evidence.

The deterministic baseline and minimal agent-enhanced workflow provide the basis for comparing expected and observed fan actions. The shared contract and safety cases provide a way to evaluate contract-shaped output and documented validation boundaries. The CSV harness records action outcomes, latency, and resource observations where available. The Raspberry Pi validation adds limited deployment evidence while keeping the thesis within the frozen scope.

4 Choice of Approach / System Design

This chapter explains the design choices that shaped the final proof-of-concept before the implementation details are presented. The purpose is to justify why the system was designed as a bounded workflow-to-action pipeline rather than as a broader automation platform, hardware project, or multi-agent system.

4.1 Design Requirements and Constraints

The system design was constrained by the need to produce a measurable and finishable thesis artifact. The central requirement was one complete event-to-action path that could be implemented, tested, and evaluated within the available thesis time frame. For this reason, the design focuses on a controlled temperature-to-fan scenario rather than multiple sensors, multiple actuators, or a general-purpose IoT platform.

The system also needed a clear separation between workflow logic and action execution. The workflow engine could decide which action should be taken, but it should not be treated as a direct hardware controller. This led to a design where n8n handles workflow orchestration, while Python middleware exposes the API boundary for action execution. This separation improves explainability because the report can distinguish event handling, decision logic, validation, middleware behavior, and simulated action execution.

Another requirement was structured communication between the workflow side and the action side. The design therefore uses JSON contracts for sensor events and action outputs instead of relying on loose text. This supports predictable routing, validation, and evaluation. The evaluation itself also needed to produce evidence that could be inspected and reused in the report, so lightweight CSV files were chosen as the main result format. Raspberry Pi validation was included only to support the edge-computing angle: it tests the middleware/action endpoint on separate hardware while avoiding a full Raspberry Pi deployment project.

4.2 Alternatives Considered

Several broader alternatives were considered but not selected because they would have shifted the focus of the thesis. A full n8n deployment on Raspberry Pi would have made the project more hardware- and deployment-oriented. That alternative may be valuable in future work, but it would also introduce operating-system, container, storage, and performance variables that are not necessary for answering

the current research questions. The selected alternative keeps n8n on the PC and validates only the middleware/action endpoint on Raspberry Pi hardware.

Another alternative was to let workflow nodes call hardware-facing actions directly. This was rejected because it would blur the safety and contract boundary. A middleware/API boundary makes the action path explicit and gives the system a single place where workflow output can be translated into controlled action execution. It also keeps future hardware integration possible without requiring workflow nodes or agent output to become direct device-control mechanisms.

The workflow design also required a choice between a deterministic-only system and a system with both deterministic and agent-enhanced paths. A deterministic-only workflow would be easier to explain and measure, but it would not test whether an LLM decision step can produce structured action output for the same event-action task. The chosen design therefore includes both: a deterministic threshold baseline and a minimal agent-enhanced workflow. The agent path is intentionally small and is not used as a basis for model benchmarking or broad multi-agent behavior.

For workflow output, loose natural-language text was rejected in favor of a shared JSON contract. Free-form text would make routing, validation, and comparison harder to report objectively. A contract-shaped action object gives the system clear expected fields and allowed values. Similarly, a full runtime safety enforcement system was rejected as too large for the bounded implementation. Instead, the thesis uses a documented validation design with allowed, blocked, and risky cases. This is sufficient for describing the intended safety boundary, while still being honest that production-grade enforcement is not implemented.

Real GPIO fan control was also considered outside the current scope. Physical hardware would add electrical and device-specific variables that are not needed to evaluate the workflow, middleware, contract, validation, and measurement pipeline. The chosen simulated fan endpoint keeps the test controlled. Finally, a heavy monitoring stack such as Prometheus or Grafana was rejected in favor of CSV-based evaluation scripts. CSV output is less sophisticated, but it is transparent, repeatable, and proportionate to the scale of the thesis.

4.3 Chosen System Design

The chosen architecture is a PC-hosted n8n workflow system with a Python middleware boundary and lightweight evaluation support. n8n runs on the PC using Docker. It receives a temperature event or test input, then executes either the deterministic workflow path or the

minimal agent-enhanced workflow path. The deterministic path uses a threshold rule. The agent-enhanced path produces structured action output that is expected to match the shared action contract.

The shared JSON contracts define the sensor-event and action-output boundary. The documented validation design describes which outputs are allowed, which malformed outputs should be blocked, and which risky or unknown outputs should not be executed directly. The Python middleware exposes the API/action endpoints used by the workflows. The evaluation harness records raw and processed CSV files so that workflow behavior, latency, and resource observations can be reported.

Limited Raspberry Pi validation is included by moving the middleware/action endpoint to Raspberry Pi hardware while n8n remains running on the PC. This design keeps the workflow engine stable and repeatable while still testing whether the PC-hosted workflow can call an edge-side endpoint over the network.

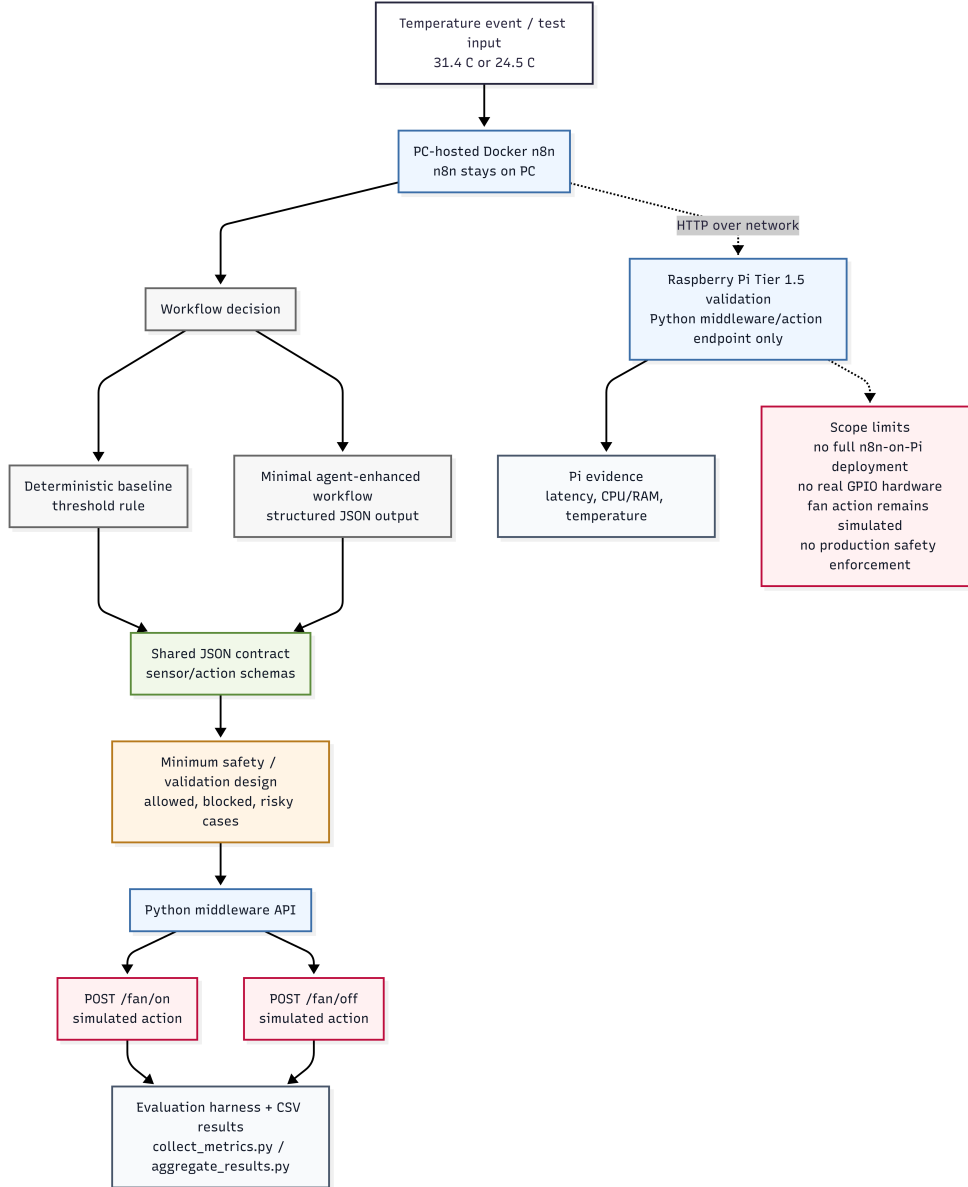


Figure 1: Final focused proof-of-concept architecture used in the thesis.

4.4 Rationale for PC-Hosted n8n and Raspberry Pi Middleware Validation

PC-hosted n8n was chosen for stability, repeatability, and reduced deployment risk. The local Docker-based setup provides a consistent workflow runtime and avoids making the thesis dependent on whether n8n itself can be optimized for Raspberry Pi hardware. This keeps the workflow comparison focused on behavior and measurement rather than on platform-specific installation issues [1], [16].

Raspberry Pi middleware validation was included for a specific purpose. It tests whether the action endpoint side can run on separate hardware and be reached from the PC-hosted workflow over the network. This supports the edge-computing motivation without turning the project into a full edge deployment or optimization study [3], [4], [5].

The design therefore does not claim n8n-on-Pi deployment. It claims only that the PC-hosted n8n workflow was designed to call a Raspberry Pi-hosted middleware/action endpoint during validation. This distinction matters because the resource and deployment implications of running n8n itself on Raspberry Pi would be different from running only the middleware endpoint.

4.5 Rationale for Deterministic and Agent-Enhanced Paths

The deterministic path provides a predictable baseline. It uses a fixed temperature threshold and therefore gives the evaluation a simple comparison point: for the defined high-temperature case, the expected action is `fan_on`; for the defined low-temperature case, the expected action is `fan_off`. This baseline is important because it separates basic workflow correctness from the additional uncertainty introduced by an LLM decision step.

The minimal agent-enhanced path was selected to test a specific and limited question: whether an LLM decision step can produce contract-shaped action output for the same controlled event-action scenario. The agent path is therefore deliberately small. It uses one prompt set, one memory choice, and the same middleware endpoints as the deterministic baseline. The comparison is about structured action output and observed behavior on the defined cases, not about broad autonomy, model ranking, or multi-agent design [2], [9].

4.6 Rationale for Middleware, Contracts, and Validation

The middleware boundary was chosen because workflow or agent output should not be treated as direct hardware control. By routing actions through Python middleware, the system has a clear API layer between decision logic and action execution. This makes it possible to describe the action path precisely and to keep future hardware integration separate from the n8n workflow design.

The JSON contracts make this boundary explicit. The sensor-event contract defines the input structure, and the action contract defines the

allowed output structure. Contract-shaped messages make the system easier to inspect and evaluate because the report can compare expected fields, allowed actions, and observed routing behavior [6].

The documented validation design adds a safety-oriented boundary around the action contract. It defines allowed output, malformed output, and risky or unknown output. This improves explainability and reportability, but it should not be confused with a fully enforced safety system. The implemented thesis artifact contains a documented validation design, not production-grade runtime safety enforcement.

4.7 Rationale for Simulated Fan Actions

The fan action was simulated to keep the proof-of-concept controlled. The relevant thesis problem is not whether a physical fan can be switched on or off; it is whether a workflow can receive an event, produce or route a contract-shaped action object, pass through a documented boundary, call middleware, and produce measurable evidence. Simulated `fan_on` and `fan_off` endpoints allow those parts of the system to be evaluated without introducing physical wiring, GPIO libraries, electrical reliability, or actuator-specific behavior.

No real GPIO hardware was controlled in the chosen design. This is a limitation, but it is also what keeps the implementation focused on workflow orchestration, middleware design, contracts, validation, and measurement. Future work could replace the simulated endpoint with physical GPIO or actuator hardware while preserving the same overall boundary between workflow logic and action execution.

4.8 Rationale for Lightweight Evaluation

The evaluation design uses CSV files because they are transparent, easy to inspect, and sufficient for the scale of the thesis. The collection script records raw observations, and the aggregation script produces processed summaries. This gives the report concrete evidence without requiring a large monitoring stack.

The scripts `collect_metrics.py` and `aggregate_results.py` support repeatable measurement of action outcomes, latency, and resource values where available. This is intentionally smaller than a full observability platform. The evaluation is thesis-scoped and small-scale; it is not designed as large-scale benchmarking across devices, models, or deployment environments.

4.9 Summary of Chosen Approach

The chosen approach gives the thesis a complete workflow-to-action proof-of-concept while keeping the system bounded and explainable. PC-hosted `n8n` provides the workflow runtime, Python middleware

provides the action boundary, shared JSON contracts make the event and action structures explicit, documented validation cases describe the safety boundary, simulated fan endpoints keep the scenario controlled, CSV files provide measurable evidence, and Raspberry Pi middleware validation adds limited separate-hardware evidence without changing the core architecture.

5 Implementation

This chapter describes the implemented proof-of-concept system. The implementation follows the narrowed Tier 1.5 scope defined for the thesis: one temperature-event path, two workflow decision modes, one shared action contract, one minimum safety and validation design, simulated fan actions, CSV-based evaluation, and Raspberry Pi validation limited to the middleware/action endpoint side. The purpose of the implementation was not to build a broad automation platform, a multi-agent architecture, or a production safety framework. Instead, the implementation creates a complete and measurable vertical slice that can be evaluated against the research questions.

5.1 System Overview

The final frozen system is organized around a temperature event that enters a workflow and is represented as a contract-shaped fan action object before reaching the middleware. At the highest level, the implemented pipeline is:

temperature event/test input → n8n workflow → workflow decision → shared contract and documented validation boundary → Python middleware → simulated `fan_on` or `fan_off` action → evaluation result files.

The architecture deliberately separates workflow logic from action execution. n8n is responsible for receiving the event and selecting an action path. The shared JSON contract describes the message shape that workflow and agent output must follow. The safety-layer files define the minimum policy for valid, malformed, and risky action output. The Python middleware then acts as the controlled API boundary for action execution. In the frozen implementation, the action endpoint changes an in-memory simulated fan state rather than controlling physical hardware.

Figure 2 shows the final report architecture. The figure also shows the Tier 1.5 Raspberry Pi validation path. In that validation, n8n remained on the PC, while only the Python middleware/action endpoint ran on the Raspberry Pi. The PC-hosted n8n workflow called the Raspberry Pi middleware over HTTP. This preserves the PC-first architecture while still testing whether the action endpoint side can run on constrained edge-style hardware.

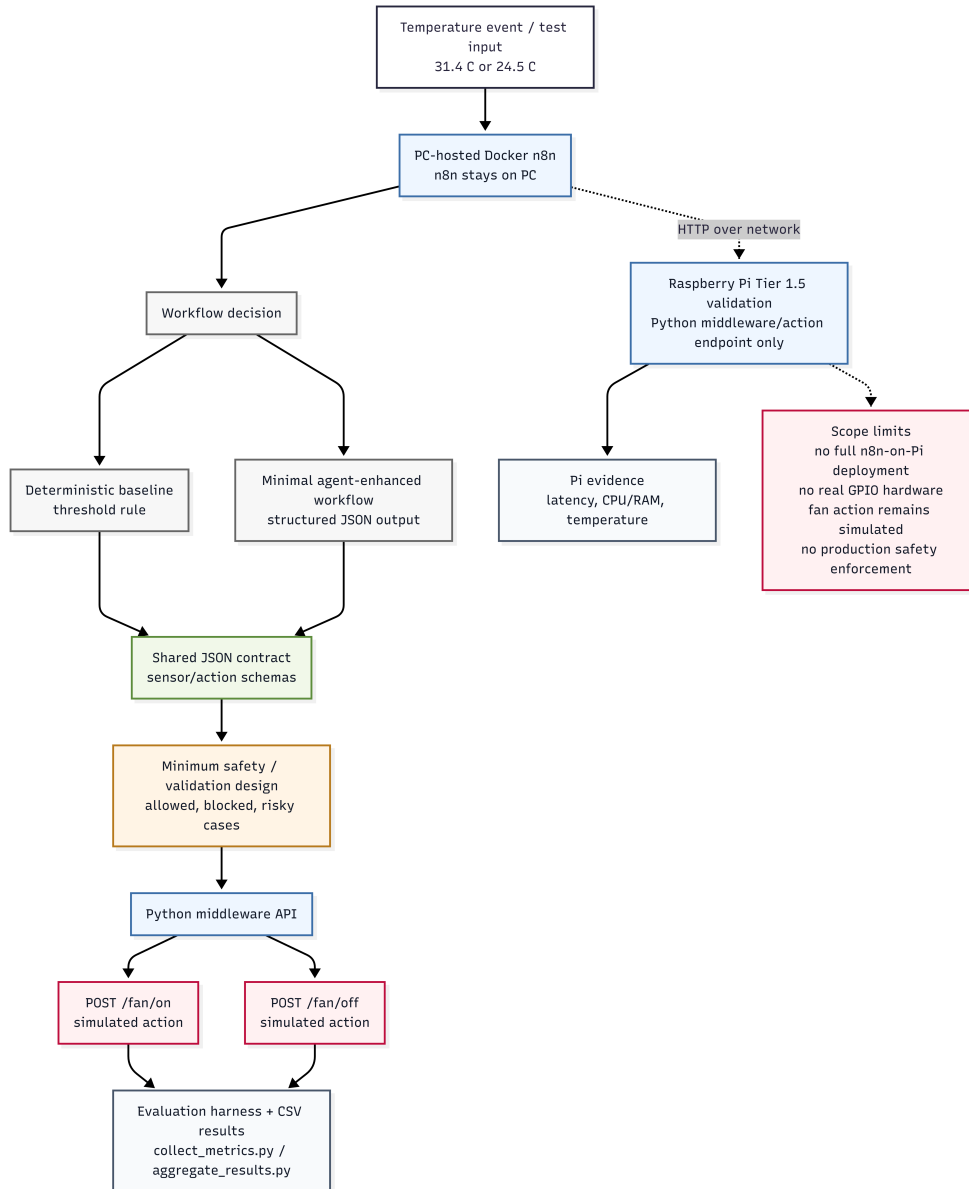


Figure 2: Frozen Tier 1.5 proof-of-concept architecture.

5.2 Local n8n Runtime with Docker

The workflow engine was implemented as a local self-hosted n8n runtime on the PC. The runtime is defined in `infrastructure/docker/docker-compose.yml` and uses the pinned container image `n8nio/n8n:1.123.37`. The compose file exposes n8n on port 5678, persists n8n data in a Docker volume, and uses local development settings such as disabled secure cookies and disabled diagnostics/version notifications [1], [16]. The PC-hosted Docker setup was chosen for stability and repeatabil-

ity. It allowed the workflow engine, middleware, contracts, and evaluation harness to be developed in a controlled local environment before adding Raspberry Pi validation. This also reduced the risk of turning the thesis into a deployment study of n8n on constrained hardware. Full n8n deployment on the Raspberry Pi was intentionally outside the implemented scope. The Raspberry Pi validation therefore appears later in the implementation as a split validation of the middleware/action endpoint side, not as a replacement for the PC-hosted n8n baseline.

The local n8n runtime is the basis for both implemented workflow modes. The deterministic baseline and the minimal agent-enhanced workflow both receive temperature events through n8n webhook flows and route their resulting fan action to the middleware. Keeping the workflow engine constant makes the two workflow modes easier to compare in the evaluation chapter.

5.3 Python Middleware API Bridge

The Python middleware implements the boundary between workflow decisions and action execution. It is placed under `middleware/` and can be started with `python -m middleware.api.app`. The HTTP request handling is implemented in `middleware/api/routes.py` using Python standard-library HTTP server components [17], while the simulated sensor and fan state are implemented in `middleware/gpio/mock_sensor.py`.

The middleware exposes four implemented endpoints. `GET /status` returns the middleware status and the current simulated hardware state. `POST /sensor-event` accepts a temperature event, normalizes it with a minimal inline check, stores it as the latest sensor event, and can forward it to n8n when a webhook URL is configured. `POST /fan/on` changes the simulated fan state to on, and `POST /fan/off` changes the simulated fan state to off. The fan endpoints return JSON responses that include the action name, the resulting fan state, a simulated flag, and a reason field.

This middleware boundary is important because it prevents workflow nodes from being treated as direct hardware controllers. Even in the Raspberry Pi validation, the action remains a simulated middleware action. No GPIO library, real fan driver, or physical actuator path is implemented in the frozen system. This keeps the implementation aligned with the thesis scope: the work evaluates an IoT-style action pipeline and contract boundary, not physical electronics control.

5.4 Shared JSON Contracts

The shared interface layer is implemented under `shared_interfaces/`. It defines the data structures that connect the middleware side and the workflow/agent side. The main reason for using structured JSON contracts was to avoid loose text as an action boundary. A free-form agent response or informal workflow payload would make routing and validation ambiguous. A small schema-based contract, by contrast, makes the expected fields explicit and gives the safety design a precise object to check [6].

The input-side schema, `shared_interfaces/json-schema/sensor-event.schema.json`, defines the temperature sensor event used by the thesis scenario. It requires `sensor_id`, `timestamp`, `type`, `value`, and `unit`. The supported sensor type is `temperature`, and the supported unit is `C`. This intentionally small input contract keeps the scenario measurable and avoids expanding into multiple sensors or device families.

The output-side schema, `shared_interfaces/json-schema/agent-action.schema.json`, defines the fan action object that workflow-produced actions should follow. It requires `action_id`, `target`, `reason`, and `requires_approval`. The allowed action identifiers are `fan_on` and `fan_off`, and the allowed target is `fan_1`. The schema also disallows additional properties. These restrictions support predictable routing: after validation, `fan_on` maps to `POST /fan/on`, and `fan_off` maps to `POST /fan/off`.

The repository also includes example payloads for a valid sensor event, a valid fan-on action, and a blocked action example. These examples support reportability and later evaluation, but they should not be interpreted as proof of production runtime enforcement. The contract layer defines the expected boundary; the safety layer defines how the boundary should be checked.

5.5 Deterministic Baseline Workflow

The deterministic baseline workflow is implemented under `cognitive_logic/workflows/asdeterministic-baseline.md` and `deterministic-baseline.json`. It is the non-AI comparison anchor for the thesis. The workflow receives a temperature event through an n8n Webhook node, reads the event value from the parsed request body, applies one threshold rule, and then calls one of the simulated fan endpoints.

The rule is intentionally simple:

if temperature $\geq$ 30.0 C, request `fan_on`; otherwise, request `fan_off`.

When the temperature is at or above the threshold, the workflow cre-

ates an action object corresponding to `fan_on` and calls `POST http://host.docker.internal:8000/fan/on`. When the temperature is below the threshold, it creates an action object corresponding to `fan_off` and calls `POST http://host.docker.internal:8000/fan/off`. The `host.docker.internal` address is used because `n8n` runs inside Docker while the middleware runs on the host during local PC execution.

The deterministic workflow exists for two reasons. First, it provides a predictable reference behavior for the same temperature-to-fan task used by the agent-enhanced workflow. Second, it gives the evaluation a simple baseline against which action correctness, consistency, and latency can be compared. The baseline does not use prompts, memory, model calls, safety approval nodes, GPIO, or Raspberry Pi deployment.

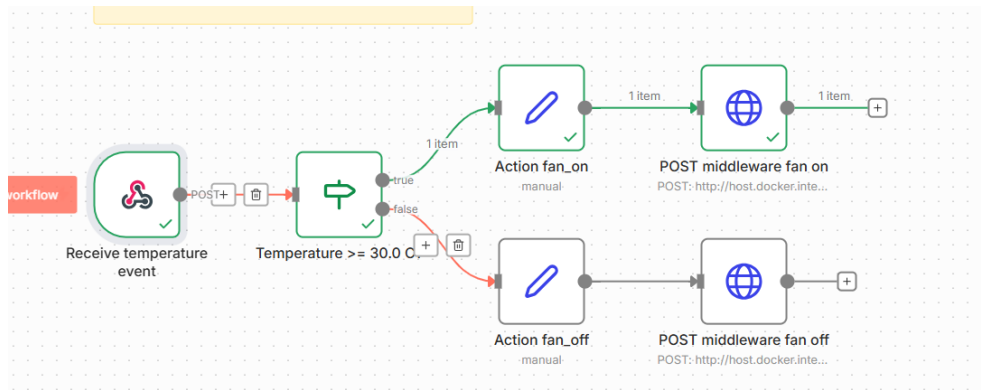


Figure 3: Deterministic baseline n8n workflow canvas.

5.6 Minimal Agent-Enhanced Workflow

The minimal agent-enhanced workflow is implemented under `cognitive_logic/workflows/` as `agent-minimal.md` and `agent-minimal.json`. It uses the same temperature-event scenario and the same middleware fan endpoints as the deterministic baseline, but it replaces the fixed threshold decision node with a constrained LLM decision step.

The workflow receives the temperature event through a Webhook node, prepares the current event as agent input, sends it to one LLM decision step, parses the returned action object, and routes supported fan actions to the middleware. The Step 7 evidence identifies the configured model node as Google Gemini Chat Model. The workflow uses one prompt set, stored in `cognitive_logic/prompts/system-prompt-v1.md`, and one memory choice, stored in `cognitive_logic/memory/memory-choice-v1.md`. The chosen memory strategy is stateless execution with no memory node connected.

The prompt constrains the LLM to return a structured JSON action object rather than a free-form explanation. The expected output follows the same action contract used elsewhere in the thesis: `action_id`, `target`, `reason`, and `requires_approval`. The workflow then parses this structured output and routes `fan_on` to `POST /fan/on` or `fan_off` to `POST /fan/off`. A minimal unrouted branch exists for output outside the expected action values, but this branch is not the formal safety layer.

The purpose of this workflow is narrow. It tests whether an agent-style decision step can produce contract-valid structured action output for the same small temperature-to-fan task as the deterministic baseline. It does not implement broad autonomy, a ReAct loop [9], a multi-agent system, model benchmarking, or persistent memory.

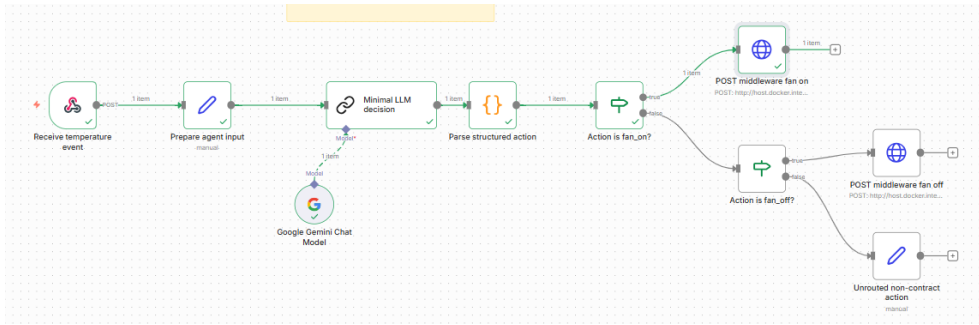


Figure 4: Minimal agent-enhanced n8n workflow canvas.

5.7 Minimum Safety and Validation Design

The safety layer is implemented as a minimum design and specification layer under `safety_layer/`. It consists of an action policy, an output validation design, a human-in-the-loop approval design, and three documented example cases. The action policy is stored in `safety_layer/policies/action-policy-v1.md`; the parser and validation design is stored in `safety_layer/parsers/output-validation-v1.md`; and the approval design is stored in `safety_layer/approvals/hit1-v1.md` [10].

The safety design is based on the shared action contract. An action is allowed to proceed without human approval only when it parses as JSON, matches the action schema, uses `fan_on` or `fan_off`, targets `fan_1`, contains a non-empty reason, has no unexpected fields, and sets `requires_approval` to false. A malformed action, an action with missing fields, an unknown action, an unknown target, or an action with unexpected fields is blocked by the policy. A risky or unknown action may be shown for human review as evidence, but policy version

1 does not allow an unknown action or target to be approved into a middleware call.

The repository includes one allowed case, one blocked malformed case, and one risky/unknown case. The allowed case describes a valid `fan_on` action targeting `fan_1`. The blocked case describes an incomplete action missing required fields. The risky case describes an invented `open_window` action targeting an unknown device, which is outside the thesis scenario and must not reach the fan endpoints.

This safety layer matters because it defines the boundary that workflow and agent output should satisfy before action execution. However, it is not a production-grade runtime safety framework. It is also not evidence that all unsafe actions are blocked by an implemented runtime gate. In the frozen implementation, it should be interpreted as the minimum validation and approval design required to make the contract boundary explicit and discussable in the report.

5.8 Evaluation Harness

The evaluation harness is implemented under `evaluation/` and `scripts/`. Its purpose is to make the proof-of-concept measurable without adding a heavy observability stack. The harness uses standard-library Python scripts and CSV files so that test execution and aggregation remain lightweight and transparent [14], [15].

The test cases are defined in `evaluation/datasets/test-cases.json`. They include deterministic workflow cases, agent workflow cases, and safety-mode cases. The metric fields are documented in `evaluation/metrics/metrics-schema.md`. Raw result files are written under `evaluation/results/raw/`, while processed summaries are written under `evaluation/results/processed/`.

The collection script, `scripts/collect_metrics.py`, posts the selected test payloads to a supplied n8n webhook URL. For workflow modes, it records the mode, test case identifier, run index, input temperature, expected action, observed action, HTTP status, success flag, latency, observed response, and error/notes fields. It also attempts to collect CPU and RAM values through `docker stats --no-stream` when Docker statistics are available [13], and it records thermal data only when a supported thermal source exists. The aggregation script, `scripts/aggregate_results.py`, reads raw CSV files and writes processed summaries for latency, resources, safety outcomes, and baseline-versus-agent comparison.

Chapter 6 reports the numerical outcomes from these CSV files. This implementation chapter therefore describes the harness structure and

the measured fields, but it does not present final latency, resource, or correctness values.

5.9 Raspberry Pi Tier 1.5 Validation Setup

The Raspberry Pi work was implemented as Tier 1.5 validation rather than full deployment. In this setup, n8n stayed on the PC and continued to run inside Docker. The Raspberry Pi ran only the Python middleware/action endpoint side. The middleware was started on the Pi with the host bound to 0.0.0.0, allowing the PC-hosted n8n workflow to call the Pi over the network using HTTP [5].

The validation reused the same temperature-to-fan scenario. High-temperature and low-temperature cases were sent through the PC-hosted n8n workflow, and the workflow called the Raspberry Pi middleware endpoints for /fan/on and /fan/off. The Pi validation evidence is stored under `evaluation/results/pi-validation/`. It includes the final workflow-run CSV, notes, startup/IP transcript, canvas and output screenshots, direct endpoint latency evidence, CPU/RAM evidence, and thermal evidence.

This split validation strengthens the edge-computing angle of the thesis without changing the core architecture. It shows that the action endpoint side of the system can be moved from the PC to the Raspberry Pi while preserving the PC-hosted workflow engine. At the same time, its limitations are important: n8n was not deployed on the Raspberry Pi, the whole system was not moved to the Pi, the fan action remained simulated, and no real GPIO hardware was controlled.

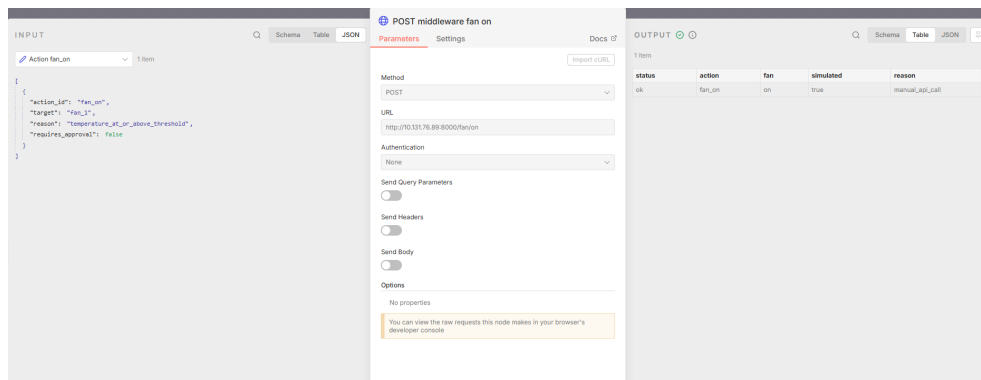


Figure 5: Raspberry Pi Tier 1.5 validation output for the high-temperature fan-on case.

5.10 Summary of Implementation

The implemented system forms one complete proof-of-concept pipeline. A temperature event can be sent into a PC-hosted n8n workflow, eval-

uated either by a deterministic threshold path or by a minimal agent-enhanced path, represented through a shared JSON action contract, bounded by a minimum validation and approval design, executed through Python middleware as a simulated fan action, and recorded through a CSV-based evaluation harness. Raspberry Pi validation extends this pipeline only by moving the middleware/action endpoint to the Pi while keeping n8n on the PC. This implementation is therefore narrow by design, but it is complete enough to support the thesis evaluation of action correctness, contract-valid agent output, latency, resource behavior, and deployment limitations.

6 Results

This chapter reports the observed results from the frozen Tier 1.5 proof-of-concept. The chapter is limited to factual reporting of the available evidence. Interpretation of what the results mean for the research questions is left to Chapter 7.

6.1 Overview of Evaluated System

The evaluated system consists of the deterministic n8n baseline workflow, the minimal agent-enhanced n8n workflow, the documented safety and validation cases, the CSV-based evaluation harness, and the Raspberry Pi Tier 1.5 validation setup. The local workflow tests used the same two temperature cases defined in `evaluation/datasets/test-cases.json`: a high-temperature case of 31.4 C, expected to produce `fan_on`, and a low-temperature case of 24.5 C, expected to produce `fan_off`.

The local workflow results are recorded in raw CSV files under `evaluation/results/raw/`. The deterministic run is stored in `run_20260429T165733Z_deterministic.csv`, and the agent-enhanced run is stored in `run_20260429T165859Z_agent.csv`. Processed summaries are stored under `evaluation/results/processed/`. Raspberry Pi validation evidence is stored under `evaluation/results/pi-validation/`.

6.2 Deterministic Baseline Results

The deterministic baseline applies the fixed threshold rule documented in the workflow evidence: temperatures greater than or equal to 30.0 C are routed to `fan_on`, and lower temperatures are routed to `fan_off`. The Step 6 runtime verification notes report that both required branches were verified locally through the n8n test webhook. The final raw deterministic CSV contains one successful row for each defined deterministic test case.

Table 3: Deterministic baseline results from `evaluation/results/raw/run_20260429T165733Z_deterministic.csv`.

Test case	Input (C)	Expected	Observed	HTTP	Latency (ms)
deterministic_high_temp	31.4	fan_on	fan_on	200	480.398
deterministic_low_temp	24.5	fan_off	fan_off	200	176.778

Both deterministic CSV rows have `success=true`. The observed middleware responses also identify the fan action as simulated. For the

high-temperature case, the response contains `action=fan_on` and `fan=on`. For the low-temperature case, the response contains `action=fan_off` and `fan=off`. These rows are the final successful deterministic raw result rows. A previous deterministic production-webhook setup attempt returned HTTP 404; that file is stored under `evaluation/results/raw/dump/` and is treated as setup evidence rather than final action-correctness evidence.

6.3 Minimal Agent-Enhanced Workflow Results

The minimal agent-enhanced workflow was evaluated with the same high- and low-temperature inputs. The Step 7 runtime verification evidence reports that the workflow was imported into n8n, connected to a Google Gemini Chat Model, and tested with both defined temperature cases. The evidence includes screenshots for the agent output, parsed action, and resulting middleware fan action. The workflow was configured for stateless execution with no memory node.

For the high-temperature case, the evidence reports that the agent selected `fan_on`, produced structured action output containing `action_id=fan_on`, and routed the parsed action to the `/fan/on` middleware endpoint. For the low-temperature case, the evidence reports that the agent selected `fan_off`, produced structured action output containing `action_id=fan_off`, and routed the parsed action to the `/fan/off` middleware endpoint. The raw agent CSV records the observed middleware actions and response status for these two cases.

Table 4: Minimal agent-enhanced workflow results from `evaluation/results/raw/run_20260429T165859Z_agent.csv`.

Test case	Input (C)	Expected	Observed	HTTP	Latency (ms)
agent_high_temp	31.4	fan_on	fan_on	200	2668.036
agent_low_temp	24.5	fan_off	fan_off	200	1995.094

Both agent CSV rows have `success=true`. The CSV file records the final observed middleware action, while the screenshot evidence records the structured agent output and parsed action routing. The result therefore supports only the narrow claim that the minimal agent workflow produced contract-shaped `fan_on` and `fan_off` actions for the defined cases. It does not provide evidence of broad agent autonomy, model comparison, or multi-agent behavior.

6.4 Safety and Validation Case Results

The safety and validation evidence consists of documented cases and expected policy outcomes. It is design/specification evidence, not production-grade runtime enforcement evidence. The safety cases are stored under `safety_layer/examples/`. The Step 8 policy defines the allowed actions as `fan_on` and `fan_off`, the allowed target as `fan_1`, and the required fields as `action_id`, `target`, `reason`, and `requires_approval`.

Table 5: Documented safety and validation cases from Step 8.

Case	Input summary	Expected policy outcome	Middleware endpoint
Allowed	Complete <code>fan_on</code> action for <code>fan_1</code> ; <code>requires_approval=false</code>	Allow after validation	POST <code>/fan/on</code>
Blocked	<code>fan_on</code> action missing reason and <code>requires_approval</code>	Block	None
Risky/approval	<code>open_window</code> targeting unknown_device; <code>requires_approval=true</code>	Reject for direct execution; may be shown for review	None

The processed file `evaluation/results/processed/safety_outcomes.csv` exists, but it currently contains only the header row. It does not contain runtime safety-enforcement rows. For this reason, the safety result in this chapter is reported as documented validation-case evidence only. The repository does not currently prove a full runtime safety framework or production-grade enforcement path.

6.5 Evaluation Harness Outputs

The evaluation harness produced both raw CSV files and processed summary files. The raw CSV files contain one successful deterministic run for each deterministic case and one successful agent run for each agent case. A previous misconfigured deterministic webhook attempt is preserved in `evaluation/results/raw/dump/` and records HTTP 404 responses because the production webhook was not registered at the time of that setup attempt.

Table 6: Evaluation harness output files.

Artifact	Observed content/status
Final deterministic raw CSV	Two successful local deterministic workflow rows: high temperature and low temperature. Path: <code>evaluation/results/raw/run_20260429T165733Z_deterministic.csv</code> .
Final agent raw CSV	Two successful local agent workflow rows: high temperature and low temperature. Path: <code>evaluation/results/raw/run_20260429T165859Z_agent.csv</code> .
Non-final setup evidence	Earlier HTTP 404 webhook setup evidence; not used as final action-correctness evidence. Path: <code>evaluation/results/raw/dump/run_20260429T165702Z_deterministic.csv</code> .
Latency summary	Processed latency summary regenerated from the final raw workflow CSV files. Path: <code>evaluation/results/processed/summary_latency.csv</code> .
Resource summary	Processed resource summary exists, with local CPU/RAM fields where available and no local thermal values. Path: <code>evaluation/results/processed/summary_resources.csv</code> .
Baseline versus agent summary	Processed baseline-versus-agent summary regenerated from the final raw workflow CSV files. Path: <code>evaluation/results/processed/baseline_vs_agent.csv</code> .
Safety outcome summary	Header only; no runtime safety outcome rows. Path: <code>evaluation/results/processed/safety_outcomes.csv</code> .

The processed summaries were regenerated from the final raw CSV files after setup-failure evidence had been separated from the final raw aggregation input. The processed deterministic and agent summaries therefore contain one successful row per defined local workflow test case. The earlier deterministic 404 setup attempt remains preserved as non-final setup evidence under `evaluation/results/raw/dump/`, but it is not part of the regenerated final processed summaries.

For the local PC runs, CPU and RAM fields were available in the raw and processed CSV files through best-effort Docker statistics. Local thermal values were recorded as `not_available`, which matches the metrics schema expectation for the Windows/PC setup.

Table 7 summarizes the final full-workflow latency rows reported in this chapter. The local deterministic and local agent rows come from `evaluation/results/raw/`. The Raspberry Pi validation rows come from `evaluation/results/pi-validation/pi-workflow-run-final`.

csv. Direct Raspberry Pi endpoint latency is supporting evidence only and is not mixed with the full workflow latency values.

Table 7: Final workflow latency summary.

Source	Test case	Input (C)	Observed	Success	Latency (ms)
Local deterministic	High temperature	31.4	fan_on	true	480.398
Local deterministic	Low temperature	24.5	fan_off	true	176.778
Local agent	High temperature	31.4	fan_on	true	2668.036
Local agent	Low temperature	24.5	fan_off	true	1995.094
Raspberry Pi validation	High temperature	31.4	fan_on	true	437.003
Raspberry Pi validation	Low temperature	24.5	fan_off	true	200.206

The internal CSV test identifiers are retained in the raw result files; the table uses human-readable labels for readability.

6.6 Raspberry Pi Tier 1.5 Validation Results

The Raspberry Pi validation used a split deployment. n8n stayed on the PC, while the Raspberry Pi ran the Python middleware/action endpoint. The middleware was started on the Raspberry Pi with `MIDDLEWARE_HOST=0.0.0.0python3-mmiddleware.api.app`. The PC-hosted n8n production webhook then called the Raspberry Pi middleware over HTTP.

The main full-workflow Pi validation evidence is stored in `evaluation/results/pi-validation/pi-workflow-run-final.csv`. It records two successful workflow calls: one high-temperature case and one low-temperature case.

Table 8: Raspberry Pi Tier 1.5 full-workflow validation from `pi-workflow-run-final.csv`.

Test case	Input (C)	Expected	Observed	HTTP	Latency (ms)
deterministic_high_temp	31.4	fan_on	fan_on	200	437.003
deterministic_low_temp	24.5	fan_off	fan_off	200	200.206

These latency values are full workflow values for the PC-hosted n8n workflow calling the Raspberry Pi middleware. They should be kept separate from direct middleware endpoint latency. The direct endpoint measurements are supporting evidence only because they do not include n8n webhook execution or workflow routing.

The observed middleware process was `python3 -m middleware.api.app`.

Table 9: Supporting Raspberry Pi middleware observations.

Observation	Value	Interpretation
Direct fan_on end-point latency	135.44 ms	Direct middleware endpoint call, not full workflow latency
Direct fan_off end-point latency	233.27 ms	Direct middleware endpoint call, not full workflow latency
Middleware CPU usage	0.0 %	Instantaneous process observation
Middleware memory usage	0.5 %	Instantaneous process observation
Middleware RSS	21048 KB	Resident memory size of the middleware process
Calculated middleware RAM	20.95 MB	RSS converted to MB
Raspberry Pi temperature	49.6 C	Device thermal observation during validation

The full-workflow CSV retains the Step 9-style fields and records `thermal_c=not_available`. The Raspberry Pi thermal value of 49.6 C is documented separately in `pi-validation-notes.md` and the thermal screenshot evidence, which uses the Raspberry Pi `vcgencmd` utility context [18]. Figure 6 shows

the selected Raspberry Pi validation output screenshot for the high-temperature fan-on case.

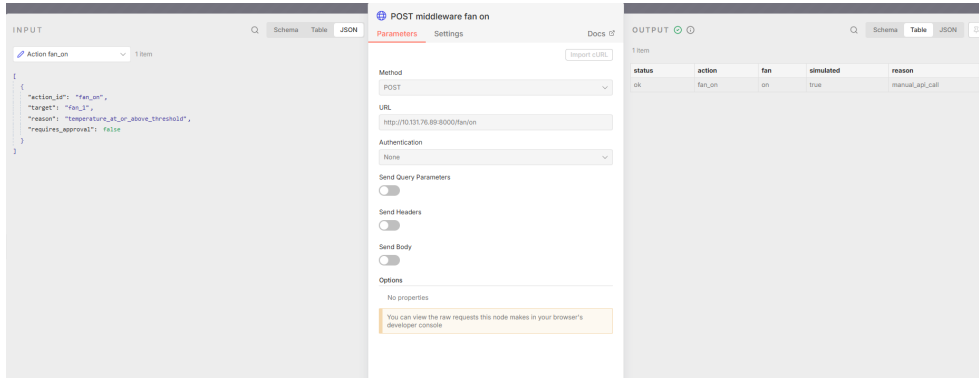


Figure 6: Raspberry Pi Tier 1.5 validation output for the high-temperature fan-on case.

The Pi validation result does not show full n8n deployment on the Raspberry Pi. It shows that the PC-hosted n8n workflow could call the Pi-hosted middleware/action endpoint for the defined high- and low-temperature cases. The fan actions remained simulated middleware actions, and no real GPIO hardware was controlled.

6.7 Summary of Results

The available results show that the deterministic baseline produced the expected `fan_on` and `fan_off` actions for the two defined temperature cases in the final raw deterministic CSV. The minimal agent-enhanced workflow produced structured, contract-shaped action output for the same defined cases according to the Step 7 screenshot evidence, and the raw agent CSV records successful `fan_on` and `fan_off` middleware responses.

The evaluation harness produced raw CSV evidence and regenerated processed summary files based on the final raw workflow evidence. The processed deterministic and agent summaries contain one successful row per defined local workflow test case, while the safety outcome summary currently contains only headers. The Raspberry Pi Tier 1.5 validation showed that PC-hosted n8n could call a Raspberry Pi-hosted middleware/action endpoint for the two deterministic temperature cases. These results are limited in scale and should not be treated as broad benchmarking.

7 Discussion

This chapter interprets the results from Chapter 6 in relation to the research questions and the design choices described in Chapters 3–5. The discussion is intentionally bounded by the defined proof-of-concept scope. The thesis evaluates a small workflow-to-action architecture, not a production IoT platform, a full Raspberry Pi deployment, or a broad agent system.

7.1 Limitations of the Current Implementation

The most important limitation is the scale of the evaluation. The system was evaluated with a small number of defined temperature cases and a small final run count. The high-temperature case used 31.4 °C and the low-temperature case used 24.5 °C. These cases are sufficient for checking whether the implemented threshold logic, agent output path, and middleware routing behave correctly for the thesis scenario, but they are not sufficient for statistical claims about reliability across many inputs, environments, or repeated deployments.

The fan action was simulated in middleware. No real GPIO sensor, physical fan, relay, or actuator was controlled. This means that the results describe workflow behavior, contract-shaped action output, middleware routing, latency measurement, and split deployment behavior. They do not describe electrical reliability, physical actuation, sensor calibration, or hardware safety. This limitation is central to the interpretation of the thesis: the contribution is the measured event-to-action pipeline, not a completed physical control system.

The Raspberry Pi validation is also limited. n8n was not deployed on the Raspberry Pi. Instead, n8n stayed on the PC and the Raspberry Pi ran only the Python middleware/action endpoint. The Pi evidence therefore shows that the PC-hosted workflow could call a Pi-hosted endpoint over the network for the defined cases. It does not show full n8n-on-Pi performance, whole-system edge deployment, or Raspberry Pi optimization.

The safety layer must be interpreted with the same care. The repository contains a documented validation design with allowed, blocked, and risky cases, but it does not prove production-grade runtime enforcement. The processed `safety_outcomes.csv` file currently contains only headers, so the safety material is design and specification evidence rather than a measured safety framework. This is still valuable for the thesis because it defines the boundary that workflow and agent output should satisfy, but it should not be described as opera-

tional safety assurance.

Finally, the agent-enhanced workflow is deliberately minimal. It uses one prompt setup, one configured model path, and stateless execution. It is not a broad multi-agent architecture, not a model benchmark, and not an autonomous planning system. The result should therefore be read as evidence about contract-shaped action output in one small workflow scenario, not as evidence about general LLM agent reliability.

7.2 Discussion of RQ1: Deterministic Workflow Accuracy and Consistency

RQ1 asks how accurately and consistently the deterministic workflow produces the expected fan action across the defined temperature test cases. The deterministic baseline behaved as expected for the final high- and low-temperature cases reported in Table 3. The high-temperature input was routed to `fan_on`, and the low-temperature input was routed to `fan_off`. This supports the conclusion that the deterministic workflow is correct for the defined cases.

The result is unsurprising but important. A deterministic threshold workflow is transparent: the rule is visible, the threshold is fixed, and the route from input value to action endpoint can be inspected. This makes the deterministic path useful as the baseline for the rest of the thesis. It verifies that the middleware endpoints, webhook path, and simulated fan responses can support the temperature-to-fan scenario before introducing an LLM decision step.

At the same time, the result should not be generalized beyond the tested cases. The evaluation does not include boundary testing around exactly 30.0 C, noisy input streams, malformed temperature events, long-duration operation, or concurrent workflow executions. The deterministic evidence therefore answers RQ1 only within the defined proof-of-concept scenario. Within that scenario, the workflow produced the expected actions; outside it, further testing would be required.

7.3 Discussion of RQ2: Agent-Enhanced Contract-Valid Output

RQ2 asks how accurately and consistently the minimal agent-enhanced workflow produces contract-valid action output compared with the deterministic baseline. The agent-enhanced workflow produced the expected `fan_on` and `fan_off` middleware actions for the same two defined temperature cases, as reported in Table 4. The screenshot ev-

idence also shows that the workflow produced structured action output and parsed the action before routing it to the middleware endpoint.

The main value of the agent-enhanced path is therefore not that it outperforms the deterministic baseline. In this simple temperature scenario, the deterministic rule is easier to explain and is sufficient for the task. The value of the agent path is that it demonstrates how an LLM decision step can be constrained to fit into the same contract-shaped action pipeline. In other words, the agent output is not treated as free text that directly controls an action. It is expected to become an action object with fields such as `action_id`, `target`, `reason`, and `requires_approval`, which can then be parsed, inspected, and routed.

This is consistent with the design rationale in Chapter 4: the agent is introduced as a limited decision component, not as a general autonomous system. Work on LLM reasoning and action shows why action-producing language models are relevant to workflow systems [9], but the thesis uses only a much smaller pattern. It does not implement a ReAct loop, multiple tools, persistent memory, or multi-agent coordination.

The limitations of RQ2 are therefore substantial. The evidence is based on one prompt setup, one configured model path, and the same two temperature cases as the deterministic baseline. There is no model comparison, no adversarial prompting, no larger test set, and no repeated-run analysis that could support broader consistency claims. The results support the narrower claim that the agent-enhanced workflow can produce contract-shaped action output for the defined cases, while leaving broader agent reliability as future work.

7.4 Discussion of RQ3: Latency, Resource, and Deployment Behavior

RQ3 asks what latency, resource, and deployment behavior is observed during local execution and Raspberry Pi middleware validation. The local evaluation harness produced raw CSV evidence for both deterministic and agent-enhanced workflow runs, and the Raspberry Pi validation produced full-workflow evidence for the PC-hosted n8n workflow calling the Pi-hosted middleware endpoint. These results show that the proof-of-concept can be measured, but they should be interpreted as limited validation evidence rather than benchmarking.

The distinction between full workflow latency and direct endpoint latency is important. The full workflow values include the PC-hosted n8n workflow path and the HTTP call to the middleware endpoint.

The direct Raspberry Pi endpoint values measure only the middleware endpoint response and do not include n8n webhook execution or workflow routing. For this reason, the direct endpoint values are useful supporting evidence, but they cannot replace the full workflow values in Table 8.

The CPU, RAM, and thermal observations also have limited scope. Local PC resource values were collected through best-effort Docker statistics where available, while local thermal values were recorded as `not_available`. The Raspberry Pi evidence includes middleware process CPU/RAM observations and a separately documented thermal value. These observations are useful because they show that the evaluation harness can capture resource-related evidence, but the small run count and limited workload mean they cannot be used to claim stable performance characteristics.

The deployment behavior is nevertheless meaningful for the thesis. The Pi validation shows that the action endpoint side can be moved to separate hardware while keeping n8n on the PC. This supports the edge-computing motivation in a limited way [3], [4]. The validation does not prove a complete edge deployment, but it demonstrates the split architecture that the thesis actually implemented.

7.5 Role of Middleware, Contracts, and Validation

The middleware boundary is one of the most important design choices in the thesis. Without it, workflow or agent output could be treated as direct action execution. By placing Python middleware between the workflow and the simulated fan action, the architecture creates a controlled place where action requests can be represented, inspected, routed, and later replaced with real hardware-specific behavior if the scope is expanded.

The JSON contracts support this boundary by reducing ambiguity. A workflow that emits loose text is difficult to evaluate objectively because the report must infer whether the text means `fan_on`, `fan_off`, or something else. A contract-shaped action object gives the system explicit fields and allowed values. JSON Schema is therefore useful not because it makes the proof-of-concept production ready, but because it gives the report a precise structure for discussing action validity [6].

The documented validation design adds a safety-oriented interpretation of the same boundary. It distinguishes allowed fan actions, malformed actions, and risky or unknown actions. This is especially important when LLM output is part of the workflow, because generated

output may be syntactically wrong, semantically unsupported, or outside the approved action set. Human-in-the-loop ideas are relevant here because risky automation often requires human review before execution [10].

However, the boundary remains a minimum design in this implementation. The safety cases document what should be allowed, blocked, or rejected for direct execution, but the thesis does not claim a production safety layer. The practical contribution is a clear contract and validation target that future runtime enforcement could implement more completely.

7.6 Broader Relevance and Possible Use Cases

The fan example is intentionally small, but the broader relevance is not the fan itself. The broader pattern is an event-validation-action pipeline: an event is received, a workflow selects or produces an action, the action is represented through a shared contract, a validation boundary limits what may proceed, middleware executes the action, and the evaluation harness records evidence. This pattern is relevant to several real-world domains, provided that future work adds the missing production and hardware layers.

In smart building control, a similar pattern could be used for ventilation, heating, lighting, or occupancy-related actions [19]. In industrial monitoring, it could support alerts or controlled responses to temperature, vibration, or process-state events [20]. In environmental sensing, the same architecture could be adapted to air quality, humidity, water-level, or greenhouse monitoring. In health and environment monitoring contexts, the pattern could support event-driven notifications or approved actions where traceability and validation are important.

These use cases were not implemented in the thesis. The current system does not include real building equipment, industrial devices, environmental sensor networks, or health-monitoring infrastructure. The fan scenario should therefore be understood as a controlled proxy. It shows how the architecture could be extended, not that those extensions have already been validated.

7.7 Ethical and Societal Considerations

Automated and AI-assisted workflows become more sensitive when their outputs can trigger physical actions. Even a simple fan example illustrates the issue: once a workflow decision becomes an action request, the system needs clear boundaries around what is allowed to happen. This becomes more important in domains such as buildings, industrial systems, environmental monitoring, or health-related con-

texts, where incorrect actions may affect comfort, equipment, safety, or trust.

For that reason, validation before execution is not only a technical design choice but also an ethical one. The system should make it possible to inspect what action was requested, why it was requested, whether it matched an approved contract, and whether it should require human review. The documented validation design and human-approval case are limited, but they point toward this responsibility: risky or unknown outputs should not be executed directly simply because a workflow or agent produced them.

Transparency and traceability are also important. CSV result files, workflow screenshots, action contracts, and documented safety cases make the proof-of-concept easier to audit than a system that only performs actions silently. This is especially relevant when AI tools are used in engineering work. The use of AI-assisted writing or workflow design should remain visible, and generated suggestions should be checked against the repository evidence rather than treated as authoritative.

7.8 Future Work

The most direct future extension is full n8n deployment on Raspberry Pi. That work would need to evaluate installation complexity, storage behavior, container performance, resource use, and workflow stability on the Pi itself. It should be treated as a separate deployment study rather than silently merged into the current proof-of-concept.

A second extension is real GPIO sensor and fan hardware. Replacing the simulated middleware endpoints with physical input and output would make the system more realistic, but it would also require hardware safety checks, electrical validation, physical error handling, and more careful risk analysis. The current middleware boundary would still be useful because it separates workflow decisions from hardware-specific control.

The evaluation could also be strengthened through repeated measurements with a larger run count, boundary-value cases around the 30.0 C threshold, malformed input tests, longer-running execution, and clearer regenerated processed summaries. Stronger runtime safety enforcement is another important future step. The documented allowed, blocked, and risky cases should be implemented as runtime checks with logs that prove whether actions were allowed, rejected, or sent for human review.

Finally, the agent-enhanced path could be expanded cautiously. Fu-

ture work could test more realistic IoT scenarios, alternative prompts, multiple model setups, or broader agent workflows. Such work should remain contract-bound and measurable. A broader multi-agent system would be a different research scope and should only be introduced if the evaluation design is expanded accordingly.

7.9 Summary of Discussion

The results support the main thesis argument within the defined project scope. The deterministic workflow produced the expected actions for the defined cases, and the minimal agent-enhanced workflow produced structured action output that could fit into the same action pipeline. The evaluation harness produced inspectable CSV evidence, and the Raspberry Pi validation showed that the middleware/action endpoint side could run on separate hardware while n8n remained on the PC.

The contribution is therefore not the simulated fan action by itself. The contribution is the measured workflow-to-action architecture: PC-hosted n8n, Python middleware, shared JSON contracts, documented validation design, CSV-based evaluation, and limited Raspberry Pi middleware validation. The work remains bounded, but that boundedness is what allows the thesis to make a clear and honest argument.

8 Conclusions

8.1 Conclusions

This thesis set out to design, implement, and evaluate a controlled proof-of-concept for workflow-based IoT-style action orchestration. The implemented system combined PC-hosted n8n, Python middleware, shared JSON contracts, a documented validation design, CSV-based evaluation, and Raspberry Pi middleware validation. The result is a focused implementation that demonstrates a complete temperature-to-fan event-action path while keeping the scope measurable and transparent.

RQ1: How accurately and consistently does the deterministic n8n workflow produce the expected fan action across the defined temperature test cases? The deterministic workflow produced the expected fan actions for the defined high-temperature and low-temperature cases. The high-temperature case led to `fan_on`, and the low-temperature case led to `fan_off`. This behavior was predictable because the workflow used an explicit threshold rule. The conclusion is limited to the defined test cases and the small run count reported in Chapter 6; it should not be interpreted as evidence of broad performance across many sensor conditions.

RQ2: How accurately and consistently does the minimal agent-enhanced workflow produce contract-valid action output compared with the deterministic baseline? The minimal agent-enhanced workflow produced structured, contract-shaped action output for the defined cases and could be routed through the same action pipeline as the deterministic baseline. This shows that an LLM-based decision step can be placed behind the same contract-oriented boundary used by the deterministic workflow in this proof-of-concept. The result does not establish broad agent autonomy, model benchmarking, or a multi-agent architecture. It shows contract-shaped output behavior for the implemented cases.

RQ3: What latency, resource, and deployment behavior is observed during local execution and Raspberry Pi middleware validation? The CSV-based evaluation captured action outcomes and latency evidence for the local workflows. The Raspberry Pi middleware validation showed that PC-hosted n8n could call the Pi-hosted middleware/action endpoint over HTTP and receive the expected simulated fan action responses. Resource and thermal observations were also collected as supporting evidence. These observations are useful for validating the defined deployment pattern, but they are not a large-scale benchmark and do not represent a full n8n deployment on Raspberry Pi.

Overall, the thesis demonstrates a small but complete workflow-to-action architecture. The implementation did not include full n8n-on-Raspberry-Pi deployment, real GPIO sensor or fan hardware, physical fan control, production-grade safety enforcement, a broad multi-agent architecture, or large-scale benchmarking. Within the defined scope, however, it shows how workflow automation, middleware, structured contracts, validation design, and measurement can be combined into a coherent action pipeline.

8.2 Main Contributions

The main contribution of the thesis is a complete temperature-to-fan event-action proof-of-concept that connects workflow orchestration to action execution through an explicit middleware boundary. The system includes a deterministic baseline workflow, a minimal agent-enhanced workflow, shared JSON contracts, a documented validation design, and a lightweight CSV-based evaluation harness.

A second contribution is the Raspberry Pi middleware validation setup. By keeping n8n on the PC and running the Python middleware/action endpoint on separate Raspberry Pi hardware, the thesis validates the networked edge-side part of the architecture without turning the work into a full edge deployment project. This supports the broader argument that even a small IoT-style action pipeline benefits from clear boundaries, structured action messages, and measurable evidence.

8.3 Future Work

Future work could extend the proof-of-concept by deploying the full n8n stack on Raspberry Pi or another edge device and comparing that deployment with the PC-hosted workflow used in this thesis. Another natural extension would be to replace the simulated `fan_on` and `fan_off` endpoints with GPIO-connected sensor and actuator hardware, while preserving the middleware and contract boundary.

The evaluation could also be expanded with more temperature cases, repeated runs, and clearer visual summaries of latency and resource behavior. The current validation design could be developed into stronger runtime safety enforcement, including stricter schema validation, clearer approval handling, and more robust rejection of unknown or risky actions. The agent-enhanced workflow could be evaluated with additional prompts, models, and memory strategies, provided that such work remains grounded in measurable action outputs rather than broad claims about autonomy.

Finally, the architecture could be adapted to more realistic IoT contexts, such as smart building control, industrial monitoring, environ-

mental sensing, or health and environment monitoring. These extensions would need their own validation and safety analysis, but they follow naturally from the same pattern used here: receive an event, convert it into a structured action decision, validate the action boundary, execute through middleware, and record evidence.

8.4 Final Remarks

The value of this thesis is not the fan action itself. The fan scenario is a controlled proxy for a larger engineering problem: how workflow automation and minimal agent-enhanced decision making can be connected to action execution in a way that remains structured, observable, and bounded. The thesis shows that a focused workflow-to-action architecture can be implemented and evaluated with modest components, while still making the limitations of the system visible.

References

- [1] n8n. *n8n Docs*. URL: <https://docs.n8n.io/> (visited on 05/24/2026) (cit. on pp. 1, 5, 17, 22).
- [2] Wil M. P. van der Aalst et al. “Workflow Patterns”. In: *Distributed and Parallel Databases* 14.1 (2003), pp. 5–51. DOI: 10.1023/A:1022883727209 (cit. on pp. 1, 5, 18).
- [3] Mahadev Satyanarayanan. “The Emergence of Edge Computing”. In: *Computer* 50.1 (2017), pp. 30–39. DOI: 10.1109/MC.2017.9 (cit. on pp. 1, 6, 18, 40).
- [4] Luigi Atzori, Antonio Iera, and Giacomo Morabito. “The Internet of Things: A Survey”. In: *Computer Networks* 54.15 (2010), pp. 2787–2805. DOI: 10.1016/j.comnet.2010.05.010 (cit. on pp. 1, 5, 18, 40).
- [5] Raspberry Pi Ltd. *Raspberry Pi Documentation*. URL: <https://www.raspberrypi.com/documentation/> (visited on 05/24/2026) (cit. on pp. 1, 6, 18, 28).
- [6] JSON Schema. *JSON Schema Specification*. URL: <https://json-schema.org/specification> (visited on 05/24/2026) (cit. on pp. 2, 6, 19, 24, 40).
- [7] Roy Thomas Fielding. “Architectural Styles and the Design of Network-based Software Architectures”. PhD thesis. University of California, Irvine, 2000. URL: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm> (visited on 05/24/2026) (cit. on p. 6).
- [8] Roy T. Fielding, Mark Nottingham, and Julian Reschke. *HTTP Semantics*. RFC 9110. 2022. DOI: 10.17487/RFC9110. URL: <https://www.rfc-editor.org/rfc/rfc9110> (visited on 05/24/2026) (cit. on p. 6).
- [9] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=WE_vluYUL-X (visited on 05/24/2026) (cit. on pp. 7, 18, 26, 39).
- [10] Saleema Amershi et al. “Power to the People: The Role of Humans in Interactive Machine Learning”. In: *AI Magazine* 35.4 (2014), pp. 105–120. DOI: 10.1609/aimag.v35i4.2513 (cit. on pp. 8, 26, 41).

- [11] Alan R. Hevner et al. "Design Science in Information Systems Research". In: *MIS Quarterly* 28.1 (2004), pp. 75–105. DOI: 10.2307/25148625 (cit. on p. 9).
- [12] Ken Peffers et al. "A Design Science Research Methodology for Information Systems Research". In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77. DOI: 10.2753/MIS0742-122240302 (cit. on p. 9).
- [13] Docker Inc. *docker container stats*. URL: <https://docs.docker.com/reference/cli/docker/container/stats/> (visited on 05/24/2026) (cit. on pp. 11, 27).
- [14] Yakov Shafranovich. *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. RFC 4180. 2005. DOI: 10.17487/RFC4180. URL: <https://www.rfc-editor.org/rfc/rfc4180> (visited on 05/24/2026) (cit. on pp. 11, 27).
- [15] Python Software Foundation. *csv – CSV File Reading and Writing*. URL: <https://docs.python.org/3/library/csv.html> (visited on 05/24/2026) (cit. on pp. 11, 27).
- [16] Docker Inc. *Docker Docs*. URL: <https://docs.docker.com/> (visited on 05/24/2026) (cit. on pp. 17, 22).
- [17] Python Software Foundation. *http.server – HTTP Servers*. URL: <https://docs.python.org/3/library/http.server.html> (visited on 05/24/2026) (cit. on p. 23).
- [18] Raspberry Pi Ltd. *Raspberry Pi OS: vcgen cmd*. URL: <https://www.raspberrypi.com/documentation/computers/os.html#vcgen cmd> (visited on 05/24/2026) (cit. on p. 35).
- [19] Daniel Minoli, Kazem Sohraby, and Benedict Occhiogrosso. "IoT Considerations, Requirements, and Architectures for Smart Buildings: Energy Optimization and Next-Generation Building Management Systems". In: *IEEE Internet of Things Journal* 4.1 (2017), pp. 269–283. DOI: 10.1109/JIOT.2017.2647881 (cit. on p. 41).
- [20] Hansong Xu et al. "A Survey on Industrial Internet of Things: A Cyber-Physical Systems Perspective". In: *IEEE Access* 6 (2018), pp. 78238–78259. DOI: 10.1109/ACCESS.2018.2884906 (cit. on p. 41).

A Source Code and Artifacts

A.1 Repository Reference

A.2 Workflow and Screenshot Evidence

A.3 Evaluation Artifacts